pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Analyzing Text, Part 2

WEEK 4, DAY 2, PART 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

By the end of this part you can

Clean text in steps

Lower case, letters only, single spaces.

```
tokens = clean(statement)
```

Keep the words that count

Stopwords out, forms of a word together.

```
token not in stop_words
```

Work on a whole column

One line, on all 46 statements at once.

```
statements['text'].str.count(pattern)
```

Turn text into a table

A count per word, and how wording changed.

```
statements['kept'].apply(Counter)
```

Cleaning text in steps

One sentence of the statement dated 2023-11-01, through the four steps of clean()

As published: 23 pieces

```
In support of these goals, the Committee decided to maintain the target range for the federal funds rate at 5-1/4 to 5-1/2 percent.
```

Step 1: lower case — `step1 = sentence.lower()` — `=LOWER(A2)`

```
in support of these goals, the committee decided to maintain the target range for the federal funds rate at 5-1/4 to 5-1/2 percent.
```

Step 2: non-letters to spaces — `step2 = re.sub(r'[^a-z\s]', ' ', step1)` — `=REGEXREPLACE(LOWER(A2), "[^a-z\s]", " ")`

```
in support of these goals  the committee decided to maintain the target range for the federal funds rate at       to       percent
```

Step 3: one space between words — `step3 = re.sub(r'\s+', ' ', step2).strip()` — `=TRIM(B2)`

```
in support of these goals the committee decided to maintain the target range for the federal funds rate at to percent
```

Step 4: split into 21 tokens — `tokens = step3.split(' ')` — `=TEXTSPLIT(C2, " ")`

`in` `support` `of` `these` `goals` `the` `committee` `decided` `to` `maintain` `the` `target` `range` `for` `the` `federal` `funds` `rate` `at` `to` `percent`

```python
def clean(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text.split(' ')
```

Dropping the digits is a decision

The job is to count words, and 5, 1 and 4 are not words. The figures were taken from the text as published, before any cleaning.

Whole statement: 344 pieces by split(), 343 tokens.

Without REGEXREPLACE: one SUBSTITUTE for each character, nested.

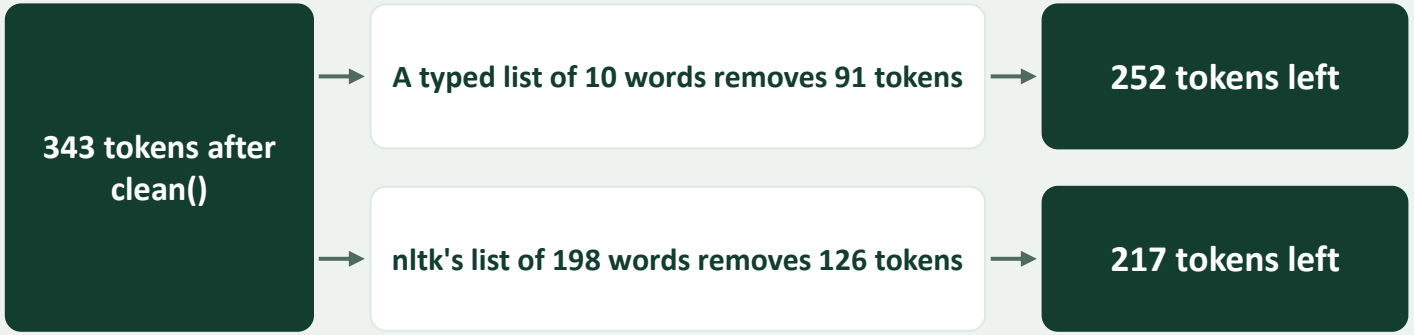
Stopwords: words every English text has

Most common tokens, statement dated 2023-11-01

All tokens	Count	After the nltk list	Count
the	29	committee	11
and	16	inflation	9
to	12	policy	7
of	11	monetary	6
committee	11	economic	5
inflation	9	percent	4

The words at the top of the first list are in any English text. They are called stopwords, and for counting they are usually removed.

The second list is a count and nothing more.



Different tokens: 180 before, 152 after the nltk list.

The single letter s, left over from Committee's, is on the nltk list.

```
short_list = ['the', 'of', 'to', 'and', 'a', 'in', 'is', 'that', 'for', 'its']
stop_words = stopwords.words('english')   # 198 words
kept = [token for token in statement_tokens if token not in stop_words]
Counter(kept).most_common(10)
```

The same in Excel: the 343 tokens in B2:B344, the stopword list in column A of a sheet named stop

`=COUNTIF(B2:B344, "the")`	returns 29	`=SUMPRODUCT(--(COUNTIF(stop!A:A, B2:B344) = 0))`	returns 217
`=COUNTIF(stop!A:A, B2) > 0`	TRUE or FALSE, filled down	`=FILTER(B2:B344, COUNTIF(stop!A:A, B2:B344) = 0)`	217 tokens

FILTER is in Excel 2021 and Microsoft 365. The SUMPRODUCT count works in every version. A pivot table of the tokens gives every count.

Stemming and its limits

Words in the statements	Stem	What happened
remain, remained, remains	remain	forms of one word, together
assess, assessing, assessments	assess	forms of one word, together
reserve, reserves	reserv	forms of one word, together
inflation	inflat	a stem is not a word
policy	polici	a stem is not a word
committee	committe	a stem is not a word
economic	econom	kept apart from economy
economy	economi	kept apart from economic
production, productivity	product	two words, merged
time, timing	time	two words, merged

The two limits: stemming can keep related words apart, and it can merge words that are not the same. A stem count is rougher than a word count. Label it as a stem count.

The Porter stemmer is a fixed list of rules about English endings. It needs no download.

152 to 146

different tokens become different stems

5

times for the stem remain: remains had 3

```
stemmer = PorterStemmer()
stems = [stemmer.stem(token) for token in kept]
```

In Excel

Excel has no function for stemming. The nearest hand tool is a wildcard.

```
=COUNTIF(B2:B344, "remain*")
```

returns 5: every token that starts with remain

Text into numbers: one line, 46 statements

A column of strings is a Series, and its string methods sit behind .str

Job	pandas, on a column of text	Excel, with the 46 texts in C2:C47	Result
Lower case	`.str.lower()`	`=LOWER(C2), filled down`	46 texts
Characters in each text	`.str.len()`	`=LEN(C2), filled down`	2,300 for 2023-11-01
Contains a piece of text	`.str.contains('balance').sum()`	`=SUMPRODUCT(--ISNUMBER(SEARCH("balance", C2:C47)))`	29 statements
Times a piece appears	`.str.count('inflation')`	`see the formula under the table`	10 for 2021-01-27
A function on every row	`.apply(clean)`	`a formula filled down a column`	16,446 tokens

`=(LEN(C2) - LEN(SUBSTITUTE(LOWER(C2), "inflation", ""))) / LEN("inflation")` returns 10 for the statement dated 2021-01-27

Whole word: .str.contains(r'\bbalance\b') is True for 25 of the 46 statements. The letters are in 29: imbalances and balances hold them.

```
statements['tokens'] = statements['text'].apply(clean)
all_counts = Counter()
for tokens in statements['kept']:
    all_counts.update(tokens)
```

16,446 tokens in the 46 statements.

10,204 after stopword removal. 507 different tokens.

Most common: committee 477, inflation 324, policy 196.

Test a pattern by a count: 46 statements, 41 found

The range pattern matched in 41 of the 46 statements, and not in 5.

In those 5 the hyphen is the non-breaking hyphen, written \u2011.

`range_pattern = r'[\d/\u2011-]+ to [\d/\u2011-]+ percent'`

With either hyphen in the class, all 46 statements have a range.

`=UNICODE(MID(C2, n, 1))` 45 plain, 8209 this one (file as UTF-8)

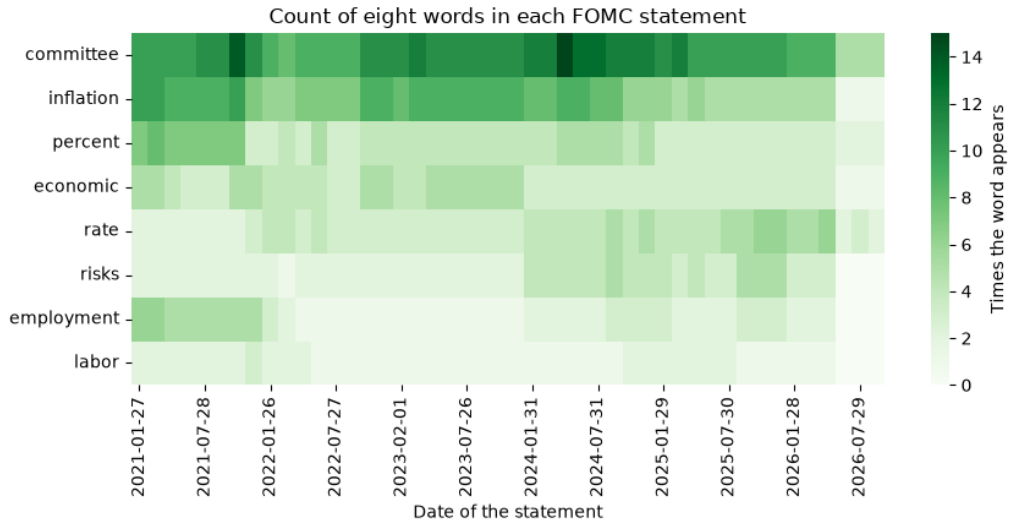
`=SUBSTITUTE(C2, UNICHAR(8209), "-")` replaces one with the other

The document-term table and its heatmap

One row for each statement, one column for each word, and each cell a count

Statement dated	committee	inflation	percent	economic	rate	risks	employment	labor
2021-01-27	10	10	7	5	2	2	6	2
2021-03-17	10	10	8	5	2	2	6	2
All 46 statements	477	324	190	163	163	121	106	62

```
counters = statements['kept'].apply(Counter)
term_table = pd.DataFrame({term:
    [counter[term] for counter in counters]
    for term in terms})
sns.heatmap(term_table.T, cmap='Greens', xticklabels=4,
    cbar_kws={'label': 'Times the word appears'},
ax=ax)
```



Mean count per 100 tokens kept, by year

Year	committee	inflation
2021	3.47	2.93
2022	4.17	3.23
2023	5.18	4.13
2024	5.94	3.64
2025	5.05	2.57
2026	5.31	1.91

A count depends on how long the document is. Per 100 tokens kept puts a long statement and a short one on the same footing.

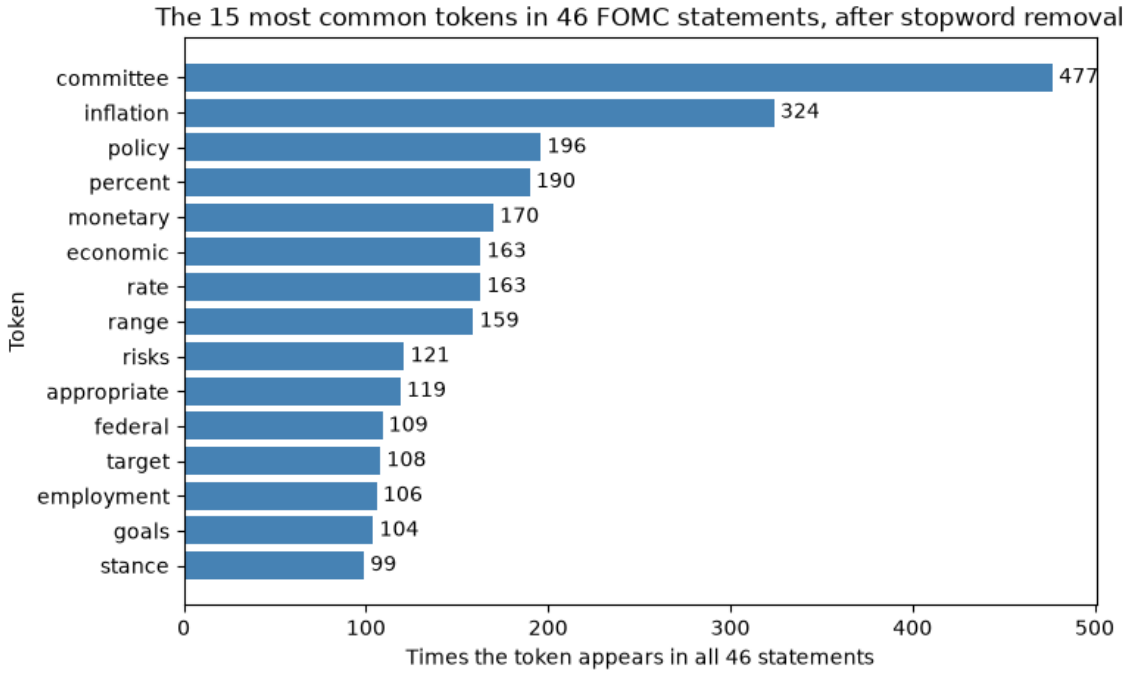
The committee row holds counts from 5 to 15, the labor row from 0 to 3.

```
per_100 = term_table.div(statements['n_kept'].values, axis=0) * 100
```

In Excel: the counts, with Home, Conditional Formatting, Color Scales.

The most common tokens and a short vocabulary

The 15 most common tokens after stopword removal



How much a short vocabulary keeps

Tokens kept	Share of the 10,204 kept tokens
10 most common	20.4 percent
20 most common	30.0 percent
50 most common	48.0 percent
All 507 different tokens	100 percent

```
full_table = pd.DataFrame(list(counters)).fillna(0).astype(int)
top_tokens = full_table.sum().nlargest(50).index
top_table = full_table[top_tokens]   # 46 rows, 50 columns
```

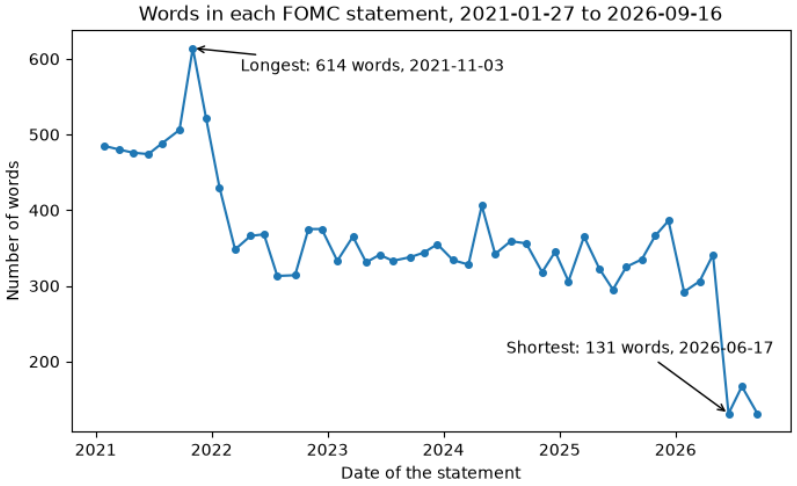
70 of the 507 tokens appear only once in the whole file.

scikit-learn's CountVectorizer(max_features=50) builds this in one step, with its own rules for what a word is.

In Excel: a pivot table of statement, token, and count.

Word count and shared wording over time

Words in each statement



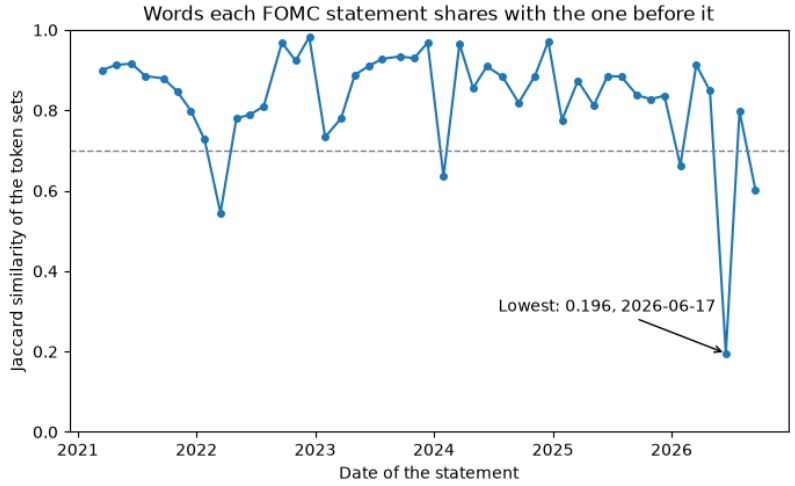
Mean 359.3 words, median 344.5.

Longest: 614 words, the statement dated 2021-11-03.

Shortest: 131 words, the statement dated 2026-06-17.

```
statements['words'] = statements['text'].str.split().str.len()
```

Words each statement shares with the one before it



`similarity = tokens in both / tokens in either`

Mean 0.832. Highest 0.982, the statement dated 2022-12-14.

Lowest 0.196, the statement dated 2026-06-17. Five are below 0.7.

```
return len(first_set & second_set) / len(first_set | second_set)
```

The charts show how long each statement is and how much wording changed. They do not say why, or what any wording means.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Clean text in steps, remove stopwords, and stem, knowing the limits
- Count tokens with Counter, and work on a column with .str and .apply()
- Spot a character that only looks right with ascii(), and widen a pattern
- Build a document-term table, and draw it as a heatmap
- Compare two documents by the words they share

Week 4, Day 3 Case Study: Statement Sentiment